

Size	Arrangement	Bubble	Insertion	Selection	Fastest
12,000	Ascending	0.00261	0.00287	7.847	Bubble
12,000	Descending	22.416	20.102	8.350	Selection
12,000	Nearly sorted	16.476	1.2462	8.294	Insertion
12,000	Random	10.318	10.1450	7.864	Selection

Analysis

Bubble sort is fastest for the ascending meaning sorted data, but both bubble and insertion sort are very similar and within margin of error distance while the size is not that large, but as array size becomes larger bubble sort clearly wins, this is due to early stopping. Insertion sort is by far the faster in nearly sorted because it minimizes both comparison checks and data movement, allowing it to run in near-linear. Selection Sort is fastest for random data as well as descending in these runs because it performs at most one main swap per pass, while Bubble Sort and Insertion Sort perform many swaps. As well as we can notice timing for selection sort is always similar no matter the case meaning its worst and best and avg case are all same. All three algorithms still have quadratic average or worst-case growth, so runtime increases quickly as the input size grows. Bubble Sort has an $O(n)$ best case because it stops when a pass makes no swaps. Its average and worst cases are $O(n^2)$. Insertion Sort has $O(n)$ best-case time and $O(n^2)$ average and worst cases. Selection Sort is $O(n^2)$ in all cases because it always scans the unsorted section. The results generally increase as the array size grows, showing quadratic increase.